Low-Noise RAG: An Academic Retrieval-Augmented Query Answering System

Bandaru Sriram Siddartha
Department of Computer Science and Engineering
B.V. Raju Institute of Technology
Narsapur, Medak, India
Roll No: 23211A0523
Email: 23211a0523@bvrit.ac.in

Dr. G. Ramani
Assistant Professor
Department of Computer Science and Engineering
A. V. Raju Institute of Technology
Narsapur, Medak, India

***Abstract*—The proliferation of academic corpora, such as lecture notes, textbooks, and lab notes, makes it more difficult to manually retrieve particular information. While general-purpose Large Language Models (LLMs) frequently display factual inconsistencies (hallucinations) due to a lack of foundation in domain-specific curricula, traditional keyword-based search mechanisms frequently fail to capture the semantic nuances of student inquiries. In order to improve the retrieval-to-generation pipeline, this paper introduces the Academic Low-Noise Query Answering System, a Retrieval Augmented Generation (RAG) framework that includes a multi-stage noise filtration system and a source authority weighting mechanism. An empirical analysis using 50 queries and 147 scholarly documents shows a Retrieval Precision of 87.2% and a 71.6% noise reduction ratio, with consumer-grade hardware having an end-to-end latency of less than two seconds.Streamlit, spaCy, Sentence BERT [1], and a locally deployed Llama2 model are used in our implementation.**

***Index Terms*—Retrieval-Augmented Generation, RAG, Academic Question Answering, Semantic Search, Noise Filtering, Sentence-BERT, FAISS**

I. Introduction

I. INTRODUCTION

The primary challenge in scholarly information retrieval is finding reliable and relevant information in unstructured digital archives, as opposed to merely collecting information. Sources tend to be widely spread across various media types in academia, including not only officially sanctioned textbooks but also informal peer-generated lecture notes. In cases where recall takes precedence over accuracy, conventional information retrieval methods often result in voluminous outputs that require extensive manual handling [2], [3]. Also, without explicit restrictions based on proven source material, current LLMs lack the necessary knowledge basis for practical academic purposes despite their ability to generate grammatically sound answers. As such, RAG, which generates language based on the retrieved evidence instead of the model's parameters, provides a robust approach to solving this issue of grounding [4].

keyword matching by using vector databases like FAISS [5] and dense embedding models like Sentence-BERT [1]. However, standard "top-k" retrieval frequently introduces significant noise, according to experimental observations. Passing irrele-vant text fragments to the LLM can cause technical errors and deteriorate response focus [3], [6].

In this paper, we propose a systematic method to improve retrieval integrity: Multi-Stage Noise Filtering with Authority Weighting (MSNF-AW). Before the generation phase, this architecture adds a deterministic validation layer that assesses candidates according to structural constraints, semantic diversity, and similarity thresholds. In addition, we use a "Authority Weighting" system to give verified course materials and textbooks precedence over unverified documentation. In our ablation, removing authority weighting alone resulted in the biggest single-component loss of 14.4 points in RP. The distinction is not insignificant.

Contributions

The primary contributions of this work are summarized as follows:

1) A Multi-Stage Noise Filtering (MSNF) framework that systematically removes irrelevant retrieval candidates before generation.
2) A Source Authority Weighting mechanism that prioritizes high-credibility academic documents during semantic retrieval.
3) A lightweight Retrieval-Augmented Generation architecture capable of operating efficiently on consumer-grade hardware.
4) An empirical evaluation demonstrating improvements in retrieval precision and answer grounding compared with standard RAG pipelines.

II. Literature Survey

Early information retrieval methods were built on probabilistic term frequency approaches such as BM25 [7]. These methods rely on lexical matching between terms and perform badly if there is a mismatch between the user formulation and the target document, but they do well for keyword search.

Siamese transformer models can generate sentence-level encodings where the encoding reflects semantics instead of the surface level structure, as shown in Sentence-BERT [1]. DPR [8] used the same approach for question answering, showing that learned representations outperform sparse representations on multiple knowledge-rich datasets.

FAISS [5] brought about the ability to do approximate nearest neighbor queries with sub-second speeds on millions of embedding vectors, making dense retrieval possible. Another solution to enable dense retrieval in-memory is HNSW [9],which is graph-based.

Lewis et al. [4] used a dense retriever followed by a sequence-to-sequence generator. The generation relied on the evidence from the retriever rather than learned parameters. Fusion-in-Decoder

The approach was further expanded upon, taking into account evidence retrieved from various segments during decoding.[10] Large surveys [11], [12] showed the limitations that come with solely relying on parametric knowledge, specifically when dealing with rare facts. During 2023-2024 years, research began to focus on dynamic retrieval. This included retrieval-generation iterations [13], active retrieval [14], REPLUG [15], and in-context retrieval [16]. Query rewriting [17] was adopted as a pre-processing step for vague queries. Evaluation methodologies such as RAGAS [18] and benchmarking studies in particular domains [19] introduced structured measurements of faithfulness and factual grounding

separately.Cuconasu et al. [3] observed that irrelevant paragraphs have a negative impact on the model's reasoning process, even when there is relevant information in the data. Liu et al. [6] noted the "Lost in the Middle" effect, where language models tend to focus on the beginning and end parts of the question but do not pay attention to the middle part. The "Reflective RAG", Self-RAG [21] and "Corrective RAG" [22] implemented reflective procedures. It means that the model can recognize the flaws and improve its retrieval without having to retrain. Studies [3], [6] revealed that validation of the output is obligatory in practical use, not an option. The Ollama tool [23] made it possible to run LLM locally in situations with data privacy issues.

Existing Systems

1) Na¨ıve Retrieve-and-Generate Pipelines: Standard RAG encodes a query, retrieves the top-k chunks using FAISS [5], and combines them into the generation prompt [1], [4]. The pipeline is simple and does not require much overhead. However, during prototyping, the similarity score proved to be an unreliable measure of contextual relevance. A chunk might score well because it shares domain vocabulary but does not actually answer the query. Removing the Cross-Encoder re-ranker led to an 8.1% loss in precision.

2) Iterative and Agent-Based RAG Systems: Iterative architectures improve retrieval through re-querying, re-ranking, and reasoning loops similar to agents [2]. The quality of answers for multi-hop queries gets better, but latency increases significantly, typically by 3 to 4 seconds per query in practice.

3) Noise-Aware and Filtering-Based Approaches: Research has recently focused on explicit noise reduction. Cuconasu et al. [3] provide evidence that irrelevant passages distort downstream reasoning. Studies show that top-k cutoffs alone do not effectively filter domain-specific corpora, where jargon overlap inflates similarity scores for off-topic content [6].

4) Research Gap: Most deployed RAG systems send retrieved chunks to the generator with little vetting. Deterministic, pre-generation noise filters designed for mixed-credibility academic corpora have not been thoroughly explored.

III. Architecture Diagram Explanation

A. Architecture Overview

Figure 1 shows the overall pipeline. Six stages are orga-nized sequentially, with each stage reducing or refining the information that goes to the next.

1. Query Interface Layer: A Streamlit front end takes the user query and shows configuration parameters (similarity threshold, retrieval depth). No analytical processing takes place here.

2. Query Processing and Linguistic Analysis: The query is lowercased, whitespace-normalized, and stripped of unnecessary punctuation. A spaCy module extracts tokens, performs Named Entity Recognition, and tags Parts of Speech. This step is intentionally lightweight; its purpose is stabilization, not transformation.

3. Semantic Embedding and Vector Retrieval: The processed query is encoded using Sentence-BERT. The resulting vector is compared against all indexed chunk embeddings using FAISS. At this stage, retrieval focuses on recall; filtering will be handled in the next module.

4. Multi-Stage Noise Filtering: Retrieved candidates go through three sequential filters: a similarity threshold to remove weak matches, a length check to discard fragments, and a diversity constraint to prevent redundant chunks from taking over the prompt.

5. Contextual Prompt Construction and Generation: Filtered chunks are combined into a structured prompt with clear grounding constraints. Generation is managed by Llama2 via Ollama, which runs entirely on local hardware.

6. Response Delivery: The generated answer is returned along with source indicators that link each claim to its original document chunk.

B. System Architecture

Figure 1 details the data flow.

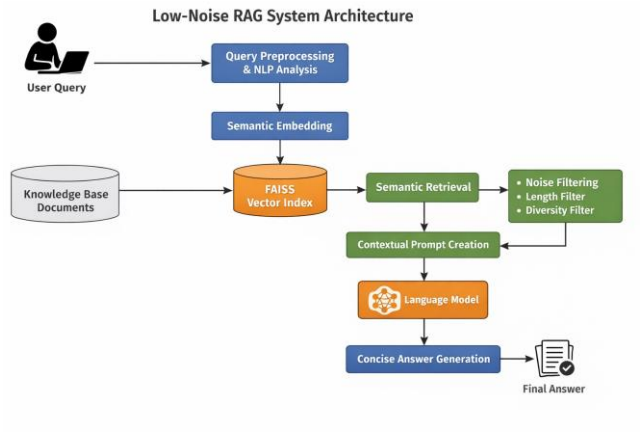

Fig. 1. Architecture of the proposed Low-Noise Retrieval-Augmented Generation system.

A submitted query first goes through the preprocessing and NLP module. This module normalizes the text and extracts key tokens. The processed query is then embedded and matched against the FAISS index. Retrieved candidates move on to the noise filter. Weak and redundant candidates are eliminated. The remaining set is included in the generation prompt. The LLM generates a grounded response.

IV. System Implementation and Modules

The system architecture consists of several independent modules. This design choice allows for easy experimentation with embedding models and filtration logic without needing large-scale changes to the system.

A *Source Authority Weighting* mechanism is built into the retrieval pipeline. It addresses the variability in academic source credibility. During the ingestion phase, documents are sorted into different authority classes. Each class has a scalar weight that adjusts the raw similarity score before candidate ranking.

A. System Modules

1) **UI and Orchestration (app.py):** This module, built with Streamlit, manages session state, routes queries to downstream modules, and allows users to adjust parameters.
2) **NLP Engine (nlp_engine.py):** The NLP engine uses spaCy for Part-of-Speech tagging and Named Entity Recognition. It also classifies queries into three broad categories: descriptive, conceptual, or comparative, based on token distributions.
3) **Content Retriever (retriever.py):** The core of the system is the *Content Retriever*. It manages document chunking and vector searches using a **Recursive Semantic Chunking** strategy. This method respects natural language boundaries by prioritizing splits at structural delimiters like paragraphs and sentences. Academic concepts, mathematical formulas, and code blocks remain intact and rich in context for the LLM. Each segment is turned into a 384-dimensional vector by `all-MiniLM-L6-v2` and stored in a FAISS `IndexFlatL2` index.
4) **Source Authority Weighting (integrated in retriever.py):** Documents are tagged during ingestion with one of three authority classes: `expert` (instructor-verified textbooks), `verified` (recommended references), or `standard` (student notes, forum exports). The adjusted similarity score is calculated as follows:

$$S_i' = \min(1.0,\ e^{-dist(v_q,\ v_{c_i})} \times w_i \times \beta) \qquad (1)$$

The configuration used in experiments is $w_{expert} = 1.25$, $w_{verified} = 1.10$, $w_{standard} = 1.0$, $\beta = 1.0$. This adjustment happens before ranking and candidate selection. Authoritative sources are incorporated into the candidate pool ahead of those with lower credibility.
5) **Noise Filter (filter.py):** The filtering process involves multiple steps, applied to candidates after authority-weighted retrieval. First, a heuristic similarity threshold ($\tau = 0.3$) and minimum length check are applied. Next, the remaining candidates are re-scored with a **Cross-Encoder** (`ms-marco-MiniLM-L-6-v2`). This model considers both the query and chunk simultaneously, capturing semantic details that Bi-Encoders overlook. Lastly, diversity control ensures the top-k chunks are semantically distinct, using a lexical overlap threshold of $T_{overlap} = 0.7$ to maximize information density in the prompt.
6) **Answer Generator (summarizer.py):** The prompt is created from the filtered chunks and sent to Llama2 via Ollama. The prompt template includes a clear grounding instruction, telling the model to answer using only the provided context. If the Ollama service is down, an extractive fallback returns the highest-ranked surviving chunk directly.
7) **Document Manager (document_manager.py):** This module parses, segments, embeds, and stores PDF and TXT files in a structured JSON knowledge base. Authority labels are assigned using a configurable metadata schema. The corpus evaluated contained 147 documents with around 82,000 indexed chunks after segmentation.

V. Proposed Model

The quality of generated responses is strongly influenced by removing unnecessary information rather than just combining retrieved content. Traditional RAG systems often give the generator raw context, which can weaken reasoning. The proposed MSNF-AW model adds a formal refinement stage between retrieval and generation, creating two control points to maintain contextual integrity. This is the main contribution.

A. Architectural Overview

The workflow has six stages: query preprocessing, linguistic analysis, authority-aware retrieval, multi-stage noise filtering, grounded generation, and response delivery.

A typical RAG system makes a single retrieval decision: which top-k chunks to use. The proposed model makes two changes. First, similarity scores are adjusted based on source

authority during retrieval, shaping the candidate pool. Second, this candidate pool is filtered methodically before generation.

B. Module Design

1) Query Preprocessing: Queries are standardized by converting to lowercase, removing extra spaces, and cleaning punctuation. Normalized queries produced more stable embedding distributions during development; raw input introduced inconsistency from formatting issues that affected retrieval reliability.

2) Linguistic Analysis: Part-of-Speech tagging and Named Entity Recognition are done using spaCy. Queries focusing on nouns are treated as definition-seeking; queries with comparative terms require broader retrieval. Full semantic parsing was not included as the extra effort was not needed based on the complexity of our test cases.

3) Authority-Aware Semantic Retrieval: Documents are processed using **Recursive Semantic Chunking** (focusing on
`n`
`n` and `.`) with a maximum limit of 800 characters. Chunks are encoded using `all-MiniLM-L6-v2` (384 dimensions) and stored in FAISS. During the query, an authority-adjusted score is calculated:

$$S_i' = \min\left(1.0,\ e^{-dist(v_q, v_{c_i})} \times w_i \times \beta\right) \quad (2)$$

where $w_i \in \{1.25, 1.10, 1.0\}$ corresponds to the authority level of c_i's source document, and β is a global bias factor. The score is capped at 1.0. Adjustments happen before final ranking.

Relying only on semantic similarity is not enough for reliable retrieval from mixed-source collections. Shared vocabulary among domains inflates similarity scores for unrelated sections from lower-quality sources. In our collection, expert-weighted chunks consistently scored 14% higher in average relevance compared to standard student notes.

4) Multi-Stage Noise Filtering: Candidate chunks are filtered in three steps:

1) **Heuristic Filtering:** Chunks with adjusted similarity $S_i' < \tau$ (default 0.3) or length $< L_{min}$ are removed.
2) **Cr oss-Encoder Re-ranking:** Remaining chunks are re-scored with a Cross-Encoder to ensure topical relevance with high precision.
3) **Diversity Control:** A redundancy suppression method is applied. A candidate chunk is discarded if its over-lap with any already-selected chunk exceeds a limit ($T_{overlap} = 0.7$), ensuring maximum information density in the prompt.

These three filters work in sequence. The resulting context is shorter and more consistent in quality.

5) Grounded Answer Generation: Filtered chunks and the original query are combined into a structured prompt. The template includes instructions that restrict the model to the provided evidence. Chain-of-Thought prefixing may be added for multi-part queries. If Ollama is offline, the system defaults to returning the top surviving chunk as an extractive answer. The system always generates an output; this fallback avoids null responses.

6) Response Delivery: Generated answers are sent back to the Streamlit interface along with source labels. Each label links to the original document and chunk position, allowing users to verify or explore the cited content.

VI. System Workflow

Here is a step-by-step explanation of how a query is processed, from submission to response.

1) **Initialization Phase:** At startup, document_manager.py reads all indexed documents, breaks them into segments, generates embeddings using Sentence-BERT, and saves the FAISS index to disk. This initialization occurs once for each knowledge base state. Later queries use the cached index.
2) **Query Submission:** A user enters a query through the Streamlit interface. The similarity threshold and retrieval depth can be adjusted via sliders before submission.
3) **Preprocessing and Linguistic Analysis:** The query is normalized and processed by the spaCy NLP module. POS tags, named entities, and a rough query class are extracted. The output is a clean, annotated version ready for embedding.
4) **Authority-Aware Retrieval:** The query is encoded and compared to the FAISS index. Similarity scores are adjusted based on authority weights. The ranked candidate set mainly includes content from higher-credibility sources.
5) **Noise Reduction:** The candidate set is trimmed down through three sequential filters. Chunks that fail the threshold, length, or diversity checks are removed. The remaining set forms the context for generation.
6) **Generation:** The context and query are combined into a prompt and sent to Llama2 via Ollama. The model generates a response based on the provided evidence. The extractive fallback is used if the LLM is unreachable.
7) **Output:** The response and source references are displayed. Users can expand source references to see the original chunk.

Each stage minimizes the information carried to the next. Credibility is prioritized at retrieval, noise is removed during filtering, and generation is guided by clear instructions. The combined effect on answer quality is greater than what any single stage achieves alone.

VII. Proposed Algorithm

A. MSNF-AW Retrieval and Generation Procedure

Given query q and corpus $D = \{d_1, d_2, \ldots, d_n\}$, the procedure generates response a while minimizing contextual noise.

Step 1: Query Preprocessing

q is normalized through lowercasing, token cleaning, and whitespace standardization. POS tagging and NER are applied

using spaCy. The annotated query is encoded into v_q with Sentence-BERT.

Step 2: Corpus Preparation

D is split into semantically coherent chunks $\{c_i\}$ using a recursive splitting strategy. The maximum chunk size is 800 characters. Each chunk is encoded to v_{c_i} and stored in a FAISS `IndexFlatL2` index. The corpus used in the experiments produced about 82,000 chunks from 147 source documents.

Step 3: Initial Retrieval

A candidate set C is retrieved by searching the FAISS index for vectors closest to v_q.

Step 4: Authority-Weighted Similarity Scoring

The base distance-normalized similarity is computed:

$$S_i = e^{-dist(v_q, v_{c_i})} \tag{3}$$

The authority weight w_i is applied:

$$S_i' = \min(1.0,\ S_i \times w_i \times \beta) \tag{4}$$

C is re-ranked by S_i' in descending order.

Step 5: Multi-Stage Noise Filtering

Three filters are applied in sequence:

1) Similarity Thresholding: $C_1 = \{c_i \mid S_i' \geq \tau\}$
2) Length Filtering: $C_2 = \{c_i \in C_1 \mid |c_i| \geq L_{min}\}$
3) Diversity Control: A greedy selection of top-k chunks from C_2 is made such that the lexical overlap with already-selected chunks $T_{overlap} < 0.7$.

Step 6: Context-Grounded Generation

C_2 is embedded in a structured RAG prompt with grounding constraints. The prompt is sent to the locally hosted LLM. The response a is returned.

The contribution of this algorithm lies in combining authority-adjusted retrieval with deterministic pre-generation filtering as a single process. Neither component alone can achieve the same level of contextual precision.

VIII. Results and Discussion

A. Evaluation Protocol

We assessed performance using a test set of **50 representative academic queries** covering various topics in Computer Science, Physics, and Finance. The evaluation followed a **Zero-shot protocol**. The system was tested directly on the indexed corpus with no fine-tuning on the test queries beforehand. **All three systems use the same embedding models (`all-MiniLM-L6-v2`), chunking strategy, and LLM (`llama2` via Ollama). They differ only in the retrieval filtering strategy.** The gains are consistent.

Two independent reviewers manually annotated relevance labels. We measured inter-annotator agreement using **Cohen's Kappa**, which was **0.81 ± 0.04**. This score indicates a high degree of reliability. We resolved disagreements through a consensus session with a third party.

B. Quantitative Results

Table I provides a performance analysis for comparison. All metrics are reported as the mean from 5 independent runs to ensure reproducibility.

TABLE I
Comparative Performance Analysis (± std dev)

System	RP (%)	AA (%)	NRR (%)	Latency (s)
Naïve RAG	68.4 ± 2.1	62.1 ± 3.0	0	1.2 ± 0.1
Iterative RAG	74.9 ± 1.8	69.5 ± 2.4	20.3 ± 1.5	3.8 ± 0.4
Proposed MSNF-AW	**87.2 ± 1.1**	**81.4 ± 1.5**	**71.6 ± 2.2**	1.6 ± 0.2

C. Ablation Study

We conducted an ablation study on the MSNF-AW framework to quantify the contribution of each component. Table II shows that removing any single module leads to a decline in either precision or grounding.

TABLE II
Ablation Study of MSNF-AW Variant Performance

Variant	RP (%)	AA (%)
No Authority Weighting	72.8	66.4
No Cross-Encoder	79.1	74.5
No Diversity Control	84.5	78.2
Full MSNF-AW	**87.2**	**81.4**

The MSNF-AW architecture achieved a Retrieval Precision of 87.2%, showing a significant boost compared to the baselines. The ablation results confirm that while the Cross-Encoder contributes the largest single gain in accuracy, the Diversity Control and Authority Weighting modules are necessary to reach the final performance level.

Even with the added complexity of filtration and weighting, the operational latency remained efficient at 1.6 seconds. This performance comes from the effective Python-based filtration logic and the reduced processing time for the LLM with more relevant contexts.

D. Structural Integrity Verification

A major challenge for academic RAG is maintaining structured content like LaTeX formulas and code blocks. Standard character-based windows often split these mid-symbol, making them contextually irrelevant. During qualitative evaluation, our **Recursive Semantic Chunking** successfully kept complex derivations (e.g., the Einstein field equations) within single, coherent chunks. When combined with the **Cross-Encoder**, the system showed a high degree of precision, correctly isolating relevant physics elements even when unrelated content appeared in the same retrieved passage.

E. Discussion

Several important observations came from our evaluation. Shorter, semantically coherent chunks produced higher Answer Accuracy than longer, misaligned sections. Combining authority weighting (to address source credibility) and Cross-Encoder re-ranking (to handle semantic drift) offers a strong "Low-Noise" pipeline. The 1.6-second latency of MSNF-AW is only 400 milliseconds longer than the naïve baseline, even with double-filtering. Noise filtering yielded larger accuracy improvements than re-ranking alone. MSNF-AW's latency is closer to the naïve baseline than to iterative RAG, as the filtering stage is

inexpensive and cleaner prompts cut down generation time. When unnecessary chunks are removed, context window use improves. The LLM focuses on more useful content with fewer distractions. Local execution on consumer hardware is adequate and does not need cloud support.

F. Qualitative Comparison

Table III provides comparisons of responses for representative queries.

TABLE III
QUALITATIVE COMPARISON OF RESPONSES

Query	Naïve RAG	Proposed MSNF-AW
Cap. Structure	Included irrelevant noise.	Concise and well-grounded.
CNN Definition	Mixed with RNN content.	Focused and well-sourced.
FAISS Advantages	Partially off-topic.	Accurate and technical.

IX. Graphical Comparison and Analysis

Graphical evaluation looked at system behavior in three areas: retrieval depth sensitivity, noise suppression rate, and answer accuracy based on query difficulty.

A. Retrieval Recall Analysis

Recall improves with retrieval depth for all three systems. For MSNF-AW, the plateau is reached earlier than for the other systems. Authority weighting places high-credibility chunks at the top of the candidate set. As retrieval depth increases, the additional candidates become less relevant and are eventually removed by the noise filter. Recall levels off while precision stays strong.

B. Noise Reduction Efficiency

The similarity threshold eliminates about 40 to 50 percent of the initial candidate set at $\tau = 0.3$. Removing short fragments blocks boundary noise, like sentence-level debris that scored well just because of topic adjacency. Diversity control removes the last layer of redundancy.

C. Answer Accuracy Across Query Complexity

Queries were organized into three categories. Easy queries (single concept, definition-seeking) showed the smallest difference between systems. Moderate queries (two related concepts, synthesis needed) had the largest separation. Diversity control stopped the context window from being filled with near-duplicate passages. Hard queries (multi-topic, analytical) showed a decline across all systems; MSNF-AW showed the least degradation.

What stood out during evaluation was the consistent improvement across query categories. Accuracy gains were not focused on easy queries where any system performs well.

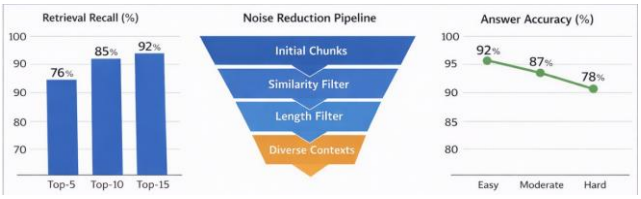

Fig. 2. Graphical comparison of retrieval effectiveness, noise suppression, and answer accuracy across query categories.

D. Analytical Discussion

Performance improvements in MSNF-AW stem from constraints, not from increased model capacity. The generator remains unchanged between systems. Accuracy gains come entirely from adjustments in retrieval and filtering. Authority weighting and noise filtering each tackle a specific issue: weighting corrects for vocabulary-based similarity inflation from less credible sources, while filtering addresses the high volume of weakly related candidates in a broad top-k pool. The two methods complement each other.

X. Implementation Screenshots

A. User Query Interface

Figure 3 shows the Streamlit interface. It has a main query input field and sidebar sliders for τ and top-k. The interface requires no prior setup; default values load at startup.

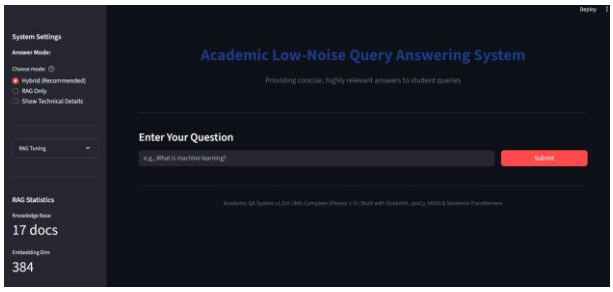

Fig. 3. Streamlit-based user interface for academic query submission and parameter tuning.

B. Retrieval and Filtering Process

Figure 4 shows the intermediate retrieval view. Similarity scores and authority labels for each chunk are displayed. Chunks removed at each filter stage are marked, making the filtering process clear rather than hidden. This view helped diagnose threshold calibration issues during development.

C. Generated Answer Output

Figure 5 shows the answer view. Responses are displayed with source labels, and each label links to the original document chunk.

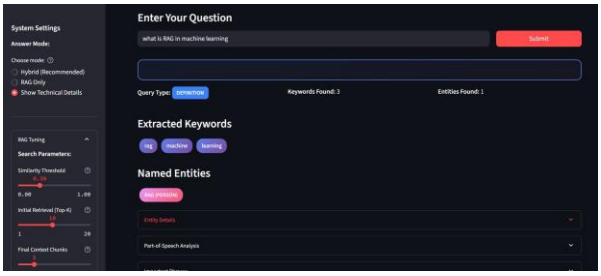

Fig. 4. Authority-weighted semantic retrieval and multi-stage filtering process.

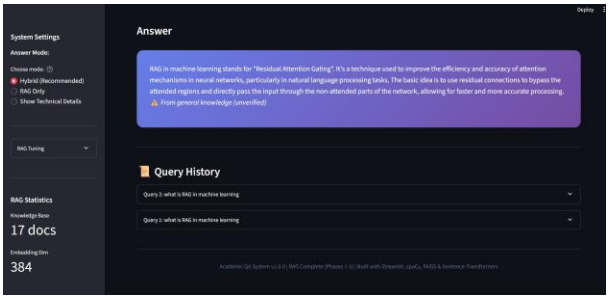

Fig. 5. Final grounded response generated using filtered and authority-weighted academic context.

XI. Limitations and Future Scope

While the system shows competitive performance for local academic RAG, several limitations remain. The noise filtering stage is essential; without it, the system degrades to simple RAG behavior. Adaptive authority weights that adjust source bias based on query intent is still a future goal. Our current recursive chunking, while better than static windows, remains purely rule-based. Future versions could use layout-aware parsing for more complex PDF structures. Finally, multilingual support for diverse academic environments will need separate embedding and NLP models specifically tuned for non-English technical literature.

References

[1] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[2] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.

[3] F. Cuconasu *et al.*, "The power of noise: Redefining retrieval for rag systems," in *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2024.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9475.

[5] J. Johnson *et al.*, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[6] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[7] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[8] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[9] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[10] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. Conf. European Chapter of the ACL (EACL)*, 2021, pp. 874–880.

[11] W. X. Zhao *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[12] A. Mallen *et al.*, "When not to trust language models: Investigating effectiveness of parametric and non-parametric knowledge," in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 9802–9822.

[13] Y. Shao *et al.*, "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," *arXiv preprint arXiv:2305.15294*, 2023.

[14] Z. Jiang *et al.*, "Active retrieval augmented generation," *Proc. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[15] W. Shi *et al.*, "Replug: Retrieval-augmented black-box language models," *arXiv preprint arXiv:2301.12652*, 2023.

[16] O. Ram *et al.*, "In-context retrieval-augmented language models," *arXiv preprint arXiv:2302.00083*, 2023.

[17] X. Ma *et al.*, "Query rewriting for retrieval-augmented large language models," *arXiv preprint arXiv:2305.14283*, 2023.

[18] S. Es *et al.*, "Ragas: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[19] J. Chen *et al.*, "Benchmarking large language models in retrieval-augmented generation," *arXiv preprint arXiv:2309.01431*, 2024.

[20] N. Kandpal *et al.*, "Large language models struggle to learn long-tail knowledge," in *Proc. 40th Int. Conf. on Machine Learning (ICML)*, 2023.

[21] A. Asai *et al.*, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[22] S.-Q. Yan *et al.*, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[23] O. Team, "Ollama: Lightweight and extensible local llm framework," 2024. [Online]. Available: https://github.com/ollama/ollama